

4.8 Flyweight Pattern

Flyweight Pattern เป็นหนึ่งในแพตเทิร์นโครงสร้างที่ออกแบบมาเพื่อประหยัดหน่วยความจำ โดยการแชร์อ็อบเจกต์ที่มีสถานะเดียวกันซ้ำ ๆ ในระบบ แทนที่จะสร้างอ็อบเจกต์ใหม่ในทุกครั้งที่ต้องการใช้งาน

:: *Flyweight Pattern* ::

ช่วยประหยัดหน่วยความจำ

ใช้งานอ็อบเจกต์ในระบบที่มีสถานะเดียวกันซ้ำ ๆ แทนที่จะสร้างใหม่ในทุกครั้งที่ต้องการใช้งาน

ตัวอย่างปัญหา

พัฒนาระบบแสดงแผนที่บ้านในหมู่บ้านจัดสรรซึ่งมีบ้านหลายหลัง การสร้างอ็อบเจกต์สำหรับบ้านแต่ละหลังจะเปลืองหน่วยความจำ

ตัวอย่าง Main program

```
House house1 = HouseFactory.getHouse("Detached");
house1.build("Location A");
House house2 = HouseFactory.getHouse("Townhouse");
house2.build("Location B");
```

โดยที่เมธอด `getHouse()` ของคลาส `House` จะตรวจสอบว่ามี house type นั้นอยู่ใน `houseMap` หรือไม่ ถ้ามีอยู่แล้วจะไม่สร้างอ็อบเจกต์ house ใหม่

```
class HouseFactory {

    private static final Map<String, House>
        houseMap = new HashMap<>();

    public static House getHouse(String houseType) {
        // ใช้ Object ที่มีอยู่แล้ว หากไม่มีจะสร้างใหม่
        House house = houseMap.get(houseType);
        if (house == null) {
            house = new ConcreteHouse(houseType);
            houseMap.put(houseType, house);
            System.out.println("Creating new house type: "
                               + houseType);
        }
        return house;
    }
}
```

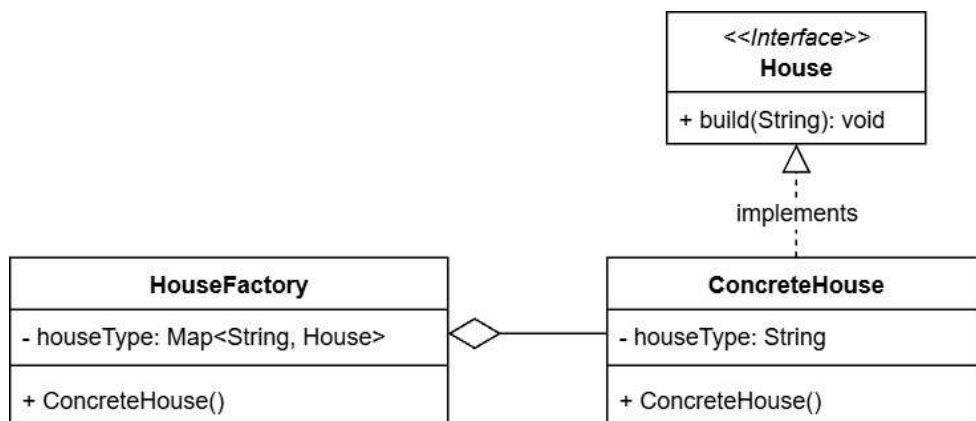
4.8.1 วัตถุประสงค์และตัวอย่าง

ความจำเป็นที่ต้องใช้ Flyweight Design Pattern

- ประหยัดหน่วยความจำในระบบที่มีอ็อบเจกต์จำนวนมาก เช่น ระบบที่ต้องแสดงองค์ประกอบที่ซ้ำกัน แต่ละอ็อบเจกต์มีสถานะบางอย่างเหมือนกัน
- เพิ่มประสิทธิภาพการทำงาน ช่วยลดจำนวนอ็อบเจกต์ที่สร้างในระบบ ทำให้ประหยัดทรัพยากรและเพิ่มความเร็ว
- แยกสถานะภายในและสถานะภายนอก Flyweight Pattern แยกสถานะออกเป็น Intrinsic State หมายถึงสถานะที่คงที่และแชร์ได้ ส่วน Extrinsic State หมายถึงสถานะที่เปลี่ยนแปลงได้และจัดการโดยผู้ใช้งาน

ตัวอย่างการใช้งาน

สมมติว่ากำลังพัฒนาระบบแสดงแผนที่บ้านในหมู่บ้านจัดสรร ซึ่งมีบ้านหลายหลัง บ้านแต่ละหลังอาจมีตำแหน่งที่ต่างกัน (สถานะภายนอก) แต่ใช้แบบบ้านเดียวกัน (สถานะภายใน) การสร้างอ็อบเจกต์สำหรับบ้านแต่ละหลังจะเปลืองหน่วยความจำ Flyweight Pattern สามารถช่วยแก้ปัญหานี้ได้



ภาพที่ xx ระบบแสดงแผนที่บ้านในหมู่บ้านจัดสรร Flyweight Pattern

House (Flyweight Interface) กำหนดโครงสร้างของ Flyweight ที่ต้องการใช้งาน เช่น การสร้างบ้าน

```
1. // Flyweight Interface
2. interface House {
3.     void build(String location);
4. }
```

ConcreteHouse (Concrete Flyweight) เก็บสถานะที่คงที่ (Intrinsic State) เช่น ประเภทของบ้าน

```
1. // Concrete Flyweight Class
2. class ConcreteHouse implements House {
3.
4.     private final String houseType; // Intrinsic State
5.
6.     public ConcreteHouse(String houseType) {
7.         this.houseType = houseType;
8.     }
9.
10.    @Override
11.    public void build(String location) { // Extrinsic State
12.        System.out.println("Building " + houseType + " at "
13.                               + location);
14.    }
15.}
```

HouseFactory (Flyweight Factory) จัดการสร้างและคืนค่าอ็อบเจกต์บ้าน หากอ็อบเจกต์แบบเดียวกันมีอยู่แล้ว จะคืนค่าอ็อบเจกต์ที่มีอยู่แทนการสร้างใหม่

```
1. // Flyweight Factory
2. class HouseFactory {
3.
4.     private static final Map<String, House>
5.         houseMap = new HashMap<>();
6.
7.     public static House getHouse(String houseType) {
8.         // ใช้ Object ที่มีอยู่แล้ว หากไม่มีจะสร้างใหม่
9.         House house = houseMap.get(houseType);
10.        if (house == null) {
11.            house = new ConcreteHouse(houseType);
12.            houseMap.put(houseType, house);
13.            System.out.println("Creating new house type: "
14.                               + houseType);
15.        }
16.        return house;
17.    }
18.}
```

Main (Client) ใช้ HouseFactory เพื่อสร้างบ้านในตำแหน่งต่าง ๆ โดยใช้อ็อบเจกต์บ้านที่แชร์สถานะเดียวกัน

```
1. // Client
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // สร้างบ้านในตำแหน่งต่าง ๆ
```

```

6.         House house1 = HouseFactory.getHouse("Detached");
7.         house1.build("Location A");
8.
9.         House house2 = HouseFactory.getHouse("Townhouse");
10.        house2.build("Location B");
11.
12.        House house3 = HouseFactory.getHouse("Detached");
13.        house3.build("Location C");
14.
15.        House house4 = HouseFactory.getHouse("Townhouse");
16.        house4.build("Location D");
17.
18.        // ตรวจสอบจำนวน Object ที่ถูกสร้าง
19.        System.out.println("\nTotal unique house types: "
20.                            + HouseFactory.houseMap.size());
21.    }
22.}

```

ผลลัพธ์

Creating new house type: Detached

Building Detached at Location A

Creating new house type: Townhouse

Building Townhouse at Location B

Building Detached at Location C

Building Townhouse at Location D

Total unique house types: 2

ข้อดีของ Flyweight Pattern

- ลดการใช้หน่วยความจำ แชนจ์อ็อบเจกต์ที่มีลักษณะเหมือนกัน แทนที่จะสร้างอ็อบเจกต์ใหม่ทุกครั้ง
- เพิ่มประสิทธิภาพของระบบ ลดการสร้างอ็อบเจกต์จำนวนมาก ช่วยเพิ่มความเร็วในการประมวลผล
- แยกสถานะภายในและสถานะภายนอก ทำให้สามารถจัดการสถานะได้ง่ายขึ้น

ข้อเสียของ Flyweight Pattern

- เพิ่มความซับซ้อนในโค้ด การแยกสถานะภายในและภายนอกอาจทำได้ซับซ้อนขึ้น

- อาจไม่เหมาะกับระบบที่ไม่มีการซ้ำของอ็อบเจกต์ หากอ็อบเจกต์ในระบบมีลักษณะเฉพาะเจาะจง อาจไม่สามารถใช้ Flyweight Pattern ได้

Flyweight Pattern เหมาะสำหรับระบบที่มีอ็อบเจกต์จำนวนมาก ที่มีสถานะเหมือนกันบางส่วน เช่น ระบบบ้านจัดสรร ระบบแผนผังบ้าน หรือระบบเกม ช่วยลดการใช้หน่วยความจำและเพิ่มประสิทธิภาพของระบบได้อย่างมาก

4.8.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Flyweight Pattern เหมาะสำหรับสถานการณ์ที่ต้องการลดการใช้หน่วยความจำในระบบที่ต้องจัดการกับอ็อบเจกต์จำนวนมาก โดยการแชร์สถานะที่เหมือนกัน ตัวอย่างปัญหาในงานจริงที่สามารถแก้ไขได้ด้วย Flyweight Pattern มีดังนี้

ตัวอย่างที่ 1 ระบบแผนที่หมู่บ้าน (Housing Map System) ในระบบแผนที่หมู่บ้าน มีการแสดงบ้านหลายร้อยหลังบนแผนที่ บ้านเหล่านี้ใช้แบบบ้านเหมือนกัน เช่น บ้านเดี่ยว, ทาวน์เฮาส์ หรือ บ้านแฝด แต่แต่ละหลังมีตำแหน่งที่แตกต่างกัน หากสร้างอ็อบเจกต์บ้านสำหรับทุกหลัง จะสิ้นเปลืองหน่วยความจำอย่างมาก การใช้ Flyweight Pattern เพื่อแชร์อ็อบเจกต์บ้านที่มีแบบเดียวกัน และจัดเก็บตำแหน่งบ้านในสถานะภายนอก (Extrinsic State)

```

1. // Flyweight Interface
2. interface House {
3.     void build(String location);
4. }

1. // Concrete Flyweight
2. class ConcreteHouse implements House {
3.
4.     private final String houseType; // Intrinsic State
5.
6.     public ConcreteHouse(String houseType) {
7.         this.houseType = houseType;
8.     }
9.
10.    @Override
11.    public void build(String location) {
12.        System.out.println("Building " + houseType + " at "
13.                            + location);
14.    }
15.}

```

```

1. // Flyweight Factory
2. class HouseFactory {
3.
4.     private static final Map<String, House> houseMap =
5.         new HashMap<>();
6.
7.     public static House getHouse(String houseType) {
8.         House house = houseMap.get(houseType);
9.         if (house == null) {
10.             house = new ConcreteHouse(houseType);
11.             houseMap.put(houseType, house);
12.             System.out.println("Creating new house type: "
13.                 + houseType);
14.         }
15.         return house;
16.     }
17. }

```

```

1. // Main
2. public class Main {
3.     public static void main(String[] args) {
4.
5.         // สร้างบ้านด้วย Flyweight Pattern
6.         House detachedHouse =
7.             HouseFactory.getHouse("Detached");
8.         detachedHouse.build("Location A");
9.
10.        House townhouse = HouseFactory.getHouse("Townhouse");
11.        townhouse.build("Location B");
12.
13.        House detachedHouse2 =
14.            HouseFactory.getHouse("Detached");
15.        detachedHouse2.build("Location C");
16.
17.        System.out.println("\nTotal unique house types: "
18.            + HouseFactory.houseMap.size());
19.    }
20. }

```

ผลลัพธ์

Creating new house type: Detached

Building Detached at Location A

Creating new house type: Townhouse

Building Townhouse at Location B

Building Detached at Location C

Total unique house types: 2

ตัวอย่างที่ 2 ระบบเกมที่มีตัวละครซ้ำกัน (Game Character System) ในเกม RPG มีการแสดงตัวละครจำนวนมาก เช่น ทหารที่มีลักษณะเหมือนกัน แต่ยืนอยู่ในตำแหน่งที่แตกต่างกัน หากสร้างอ็อบเจกต์สำหรับทุกตัวละคร จะสิ้นเปลืองหน่วยความจำอย่างมาก ใช้ Flyweight Pattern เพื่อแชร์อ็อบเจกต์ตัวละคร และจัดเก็บตำแหน่งตัวละครในสถานะภายนอก

```
1. // Flyweight Interface
2. interface Character {
3.     void render(int x, int y);
4. }

1. // Concrete Flyweight
2. class Soldier implements Character {
3.
4.     private final String role = "Soldier"; // Intrinsic State
5.
6.     @Override
7.     public void render(int x, int y) {
8.         System.out.println("Rendering " + role
9.                             + " at (" + x + ", " + y + ")");
10.    }
11.}

1. // Flyweight Factory
2. class CharacterFactory {
3.
4.     private static final Map<String, Character> characterMap
5.         = new HashMap<>();
6.
7.     public static Character getCharacter(String role) {
8.         Character character = characterMap.get(role);
9.         if (character == null) {
10.            if (role.equals("Soldier")) {
11.                character = new Soldier();
12.            }
13.            characterMap.put(role, character);
14.            System.out.println("Creating new character: "
15.                               + role);
16.        }
17.        return character;
18.    }
19.}

1. // Main
2. public class Main {
3.     public static void main(String[] args) {
4.         Character soldier1 =
5.             CharacterFactory.getCharacter("Soldier");
6.         soldier1.render(10, 20);
7.
8.         Character soldier2 =
9.             CharacterFactory.getCharacter("Soldier");
```

```

10.         soldier2.render(30, 40);
11.
12.         System.out.println("\nTotal unique characters: "
13.             + CharacterFactory.characterMap.size());
14.     }
15.}

```

ผลลัพธ์

Creating new character: Soldier

Rendering Soldier at (10, 20)

Rendering Soldier at (30, 40)

Total unique characters: 1

ตัวอย่างที่ 3 ระบบแสดงข้อความในเอกสาร (Text Rendering System) ในระบบแสดงข้อความ (Text Rendering System) เช่น โปรแกรมประมวลผลคำ หรือ ระบบการแสดงผลข้อความ แต่ละตัวอักษรที่แสดงบนหน้าจออาจเหมือนกันหลายครั้ง เช่น ตัวอักษร 'A' หากสร้างอ็อบเจกต์สำหรับแต่ละตัวอักษร จะเปลืองหน่วยความจำมาก Flyweight Pattern ช่วยให้สามารถแชร์อ็อบเจกต์ของตัวอักษรที่เหมือนกันได้ ใช้ Flyweight Pattern เพื่อแชร์อ็อบเจกต์ตัวอักษรเดียวกัน และจัดเก็บตำแหน่งหรือขนาดตัวอักษรในสถานะภายนอก

```

1. // Flyweight Interface
2. // กำหนดวิธีการแสดงผลตัวอักษร เช่น render() ที่ใช้แสดงตัวอักษรพร้อมข้อมูลเกี่ยว
3. // กับฟอนต์ ขนาด, และตำแหน่ง
4.
5. interface Glyph {
6.     void render(String font, int size, int x, int y);
7. }

1. // Concrete Flyweight Class
2. // ตัวแทนของตัวอักษรแต่ละตัว (Intrinsic State) เช่น 'H', 'E', 'L', 'O'
3. // ใช้ข้อมูลสถานะภายนอก (Extrinsic State) เช่น ฟอนต์, ขนาด, และตำแหน่ง
4. // ในการแสดงผล
5.
6. class CharacterGlyph implements Glyph {
7.
8.     private final char character; // Intrinsic State
9.
10.    public CharacterGlyph(char character) {
11.        this.character = character;
12.    }
13.
14.    @Override
15.    public void render(String font, int size, int x, int y) {
16.        System.out.println("Rendering character '" + character
17.            + "' with font '" + font + "', size " + size

```

```

18.         + " at (" + x + ", " + y + ")");
19.     }
20.}

```

```

1. // Flyweight Factory
2. // จัดการสร้างและคืนค่า Object ของตัวอักษร หาก Object สำหรับตัวอักษรนั้นมีอยู่
   แล้ว
3. // จะคืนค่า Object เดิมแทนการสร้างใหม่
4. class GlyphFactory {
5.
6.     private static final Map<Character, Glyph> glyphMap
7.         = new HashMap<>();
8.
9.     public static Glyph getGlyph(char character) {
10.         Glyph glyph = glyphMap.get(character);
11.         if (glyph == null) {
12.             glyph = new CharacterGlyph(character);
13.             glyphMap.put(character, glyph);
14.             System.out.println("Creating glyph for character: "
15.                 + character);
16.         }
17.         return glyph;
18.     }
19.}

```

```

1. // Client
2. // จำลองการแสดงผลข้อความ "HELLO" โดยใช้ Flyweight Pattern
3. // ตัวอักษร 'L' ถูกใช้ซ้ำ แต่สร้าง Object เพียงครั้งเดียว
4. public class Main {
5.     public static void main(String[] args) {
6.
7.         String text = "HELLO";
8.         String font = "Arial";
9.
10.        int x = 0, y = 10;
11.        int size = 12;
12.
13.        for (char c : text.toCharArray()) {
14.            // ใช้ Flyweight Pattern
15.            Glyph glyph = GlyphFactory.getGlyph(c);
16.            glyph.render(font, size, x, y);
17.            // ขยับตำแหน่งสำหรับตัวอักษรถัดไป
18.            x += 15;
19.        }
20.
21.        System.out.println("\nTotal unique glyphs created: "
22.            + GlyphFactory.glyphMap.size());
23.    }
24.}

```

ผลลัพธ์

Creating glyph for character: H

Rendering character 'H' with font 'Arial', size 12 at (0, 10)

Creating glyph for character: E

Rendering character 'E' with font 'Arial', size 12 at (15, 10)

Creating glyph for character: L

Rendering character 'L' with font 'Arial', size 12 at (30, 10)

Rendering character 'L' with font 'Arial', size 12 at (45, 10)

Creating glyph for character: O

Rendering character 'O' with font 'Arial', size 12 at (60, 10)

Total unique glyphs created: 4

4.9 บทสรุป

แพตเทิร์นโครงสร้าง (Structural Patterns) เป็นหนึ่งในสามประเภทหลักของ Design Patterns ที่ระบุไว้ในหนังสือ "Design Patterns: Elements of Reusable Object-Oriented Software" โดย Gang of Four (GoF) ซึ่งประกอบด้วย Erich Gamma, Richard Helm, Ralph Johnson, และ John Vlissides แพตเทิร์นเหล่านี้ถูกพัฒนาขึ้นเพื่อแก้ปัญหการออกแบบโครงสร้างของโปรแกรม โดยเฉพาะการจัดระเบียบและเพิ่มความสัมพันธ์ระหว่าง Class และ Object ในระบบ มีวัตถุประสงค์ ได้แก่

- เพิ่มความยืดหยุ่นในโครงสร้างของโปรแกรม แพตเทิร์นโครงสร้างช่วยสร้างความสัมพันธ์ที่ยืดหยุ่นระหว่าง Class และ Object โดยเน้นการออกแบบที่ง่ายต่อการขยายในอนาคต
- ลดความซับซ้อนของระบบ ช่วยลดการพึ่งพา (Dependency) ระหว่าง Class และเพิ่มความสามารถในการนำโค้ดกลับมาใช้ใหม่ (Reusability)
- ซ่อนรายละเอียดการทำงาน ใช้เพื่อซ่อนความซับซ้อนของ Subsystem ทำให้โค้ดดูสะอาดและเข้าใจง่าย
- สนับสนุนการทำงานร่วมกันของส่วนประกอบที่แตกต่างกัน จัดการการเชื่อมต่อระหว่างโมดูลหรือองค์ประกอบที่มีโครงสร้างแตกต่างกัน เช่น การเชื่อมต่อ Legacy Code กับระบบใหม่

แพตเทิร์นโครงสร้างประกอบด้วย 7 รูปแบบหลัก ซึ่งแต่ละแบบมีวัตถุประสงค์และการนำไปใช้งานที่แตกต่างกัน

- 1) Adapter Pattern ใช้เพื่อแปลงอินเทอร์เฟซของคลาส หรืออ็อบเจกต์ ให้สามารถทำงานร่วมกับอินเทอร์เฟซอื่นได้ มีวัตถุประสงค์เพื่อช่วยให้ระบบที่มีโครงสร้างหรืออินเทอร์เฟซแตกต่างกันสามารถทำงานร่วมกันได้ ตัวอย่างเช่นการเชื่อมต่อระบบเก่ากับระบบใหม่

- 2) Bridge Pattern ใช้เพื่อแยก Abstraction และ Implementation ออกจากกัน มีวัตถุประสงค์เพื่อเพิ่มความยืดหยุ่นในการขยายทั้งฝั่ง Abstraction และ Implementation โดยไม่กระทบกัน ตัวอย่างเช่น การจัดการชนิดของบ้านและวัสดุในการสร้าง
- 3) Composite Pattern ใช้จัดการโครงสร้างแบบลำดับชั้นที่มีองค์ประกอบแบบเดี่ยวและแบบกลุ่ม มีวัตถุประสงค์เพื่อช่วยให้สามารถจัดการอ็อบเจกต์เดี่ยว และอ็อบเจกต์กลุ่มด้วยอินเทอร์เฟซเดียวกัน ตัวอย่างเช่น โครงสร้างบ้านที่มีห้องและเฟอร์นิเจอร์
- 4) Decorator Pattern ใช้สำหรับเพิ่มฟังก์ชันใหม่ให้กับอ็อบเจกต์แบบ Dynamic โดยไม่ต้องเปลี่ยนแปลงโครงสร้างเดิม มีวัตถุประสงค์เพื่อเพิ่มความสามารถของอ็อบเจกต์ในระหว่าง Runtime ตัวอย่างเช่น เพิ่มระบบไฟฟ้าอัจฉริยะหรือระบบรักษาความปลอดภัยในบ้าน
- 5) Facade Pattern ใช้เพื่อสร้างอินเทอร์เฟซที่ง่ายและลดความซับซ้อนของการใช้งานระบบที่มี Subsystem หลายตัว มีวัตถุประสงค์เพื่อซ่อนรายละเอียดภายในของระบบและเพิ่มความสะดวกในการใช้งาน ตัวอย่างเช่น ควบคุมบ้านอัจฉริยะ เช่น เปิดไฟ ล็อกประตู และเปิดแอร์ ด้วยคำสั่งเดียว
- 6) Flyweight Pattern ใช้เพื่อแชร์อ็อบเจกต์ที่มีสถานะเดียวกันในระบบที่มีอ็อบเจกต์จำนวนมาก มีวัตถุประสงค์เพื่อลดการใช้หน่วยความจำและเพิ่มประสิทธิภาพของระบบ ตัวอย่างเช่น แสดงแผนที่บ้านในหมู่บ้านจัดสรร โดยใช้ อ็อบเจกต์บ้านแบบเดียวกันซ้ำ ๆ
- 7) Proxy Pattern ใช้สร้างตัวแทนสำหรับอ็อบเจกต์จริงเพื่อควบคุมหรือเพิ่มฟังก์ชันเสริมก่อนการเข้าถึง มีวัตถุประสงค์เพื่อควบคุมการเข้าถึง เพิ่ม Lazy Loading หรือจัดการการเข้าถึงผ่านเครือข่าย ตัวอย่างเช่น ระบบล็อกบ้านอัจฉริยะที่ตรวจสอบสิทธิ์ผู้ใช้อีก่อนปลดล็อก

แพตเทิร์นโครงสร้างช่วยให้ระบบมีความ ยืดหยุ่น, ลดความซับซ้อน, และ ง่ายต่อการบำรุงรักษา ช่วยให้การออกแบบเชิงวัตถุมีความสอดคล้องกับหลักการ SOLID เช่น Single Responsibility และ Open-Closed Principle และใช้ได้ในงานหลากหลาย เช่น ระบบบ้านอัจฉริยะ การเชื่อมต่อระบบเก่า-ใหม่ หรือการแสดงผลข้อมูลขนาดใหญ่

4.10 คำถามท้ายบท

- 1) อธิบายวัตถุประสงค์หลักของ Structural Patterns ว่าคืออะไร
- 2) Adapter Pattern ใช้แก้ปัญหาอะไร และให้ตัวอย่างการใช้งาน
- 3) Bridge Pattern แตกต่างจาก Adapter Pattern อย่างไร
- 4) จงยกตัวอย่างสถานการณ์ที่เหมาะสมกับการใช้ Composite Pattern
- 5) อธิบายความแตกต่างระหว่าง Decorator Pattern และ Composite Pattern

- 6) ทำไมถึงควรใช้ Facade Pattern ในระบบที่มีหลาย Subsystems
- 7) Flyweight Pattern เหมาะกับสถานการณ์แบบใด
- 8) Proxy Pattern ใช้แก้ปัญหาอะไรในระบบบ้านอัจฉริยะ
- 9) จงอธิบายการแยก Intrinsic State และ Extrinsic State ใน Flyweight Pattern พร้อมยกตัวอย่าง
- 10) จงยกตัวอย่างสถานการณ์ที่เหมาะสมกับการใช้ Bridge Pattern

4.11 เอกสารอ้างอิง

- 1) Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- 2) Freeman, E., & Freeman, E. (2004). Head First Design Patterns. O'Reilly Media.
- 3) Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.
- 4) Shalloway, A., & Trott, J. R. (2004). Design Patterns Explained: A New Perspective on Object-Oriented Design. Addison-Wesley.
- 5) Geary, D. M., & Horstmann, C. S. (2008). Core Java, Volume II: Advanced Features. Prentice Hall.
- 6) Bates, B., & Sierra, K. (2017). Head First Java (3rd Edition). O'Reilly Media.
- 7) Alur, D., Crupi, J., & Malks, D. (2003). Core J2EE Patterns: Best Practices and Design Strategies. Prentice Hall.
- 8) Eckel, B. (2006). Thinking in Java (4th Edition). Prentice Hall.
- 9) Owen, J. (2011). Professional Java Design Patterns. Wrox.